Lab Manual 15: File Handling & Database Connectivity in Java with SQL Server

1. Introduction

This lab manual covers the complete implementation and understanding of File Handling and Database Connectivity in Java using SQL Server. Students will learn how to connect a Java application to a SQL Server database using JDBC, perform CRUD operations, and handle real-world data storage needs.

2. Prerequisites

1. Java installed (Java SE Development Kit)
2. IDE
3. Microsoft SQL Server
4. SQL Server Management Studio (SSMS)
5. Microsoft JDBC Driver for SQL Server (.jar file)

3. SQL Server Setup

1. Open SQL Server Management Studio (SSMS).

2. Connect to server using:

   1. Server Name: localhost
   2. Authentication: SQL Server Authentication
   3. Username: sa
   4. Password: [your-password]

3. Execute the following SQL script to create database and table:

```
CREATE DATABASE College;
GO
USE College;
GO
CREATE TABLE Students (
```

```
    id INT IDENTITY(1,1) PRIMARY KEY,
    name VARCHAR(100),
    age INT,
    department VARCHAR(50)
);
GO
```

4. Adding JDBC Driver to Java Project

1. Download the Microsoft JDBC Driver for SQL Server.

2. Extract and locate the 'mssql-jdbc-<version>.jar' file.

3. Add the JAR file to your Java project via your IDE (Project > Build Path > Add External JARs).

5. Java Code to Connect to SQL Server

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnect {
    public static void main(String[] args) {
        // Connection URL format for SQL Server
        String url = "jdbc:sqlserver://localhost:1433;databaseName=College";
        String user = "sa";        // SQL Server username
        String password = "123456"; // Your SQL Server password

        try {
            // Register JDBC driver (optional for latest versions)
            Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");

            // Create a connection to the database
            Connection con = DriverManager.getConnection(url, user, password);

            // If connection is successful, print message
            System.out.println("    Connection successful to SQL Server database!");

            // Close connection
            con.close();
        } catch (ClassNotFoundException e) {
```

```
            System.out.println("    JDBC Driver not found: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("    Connection failed: " + e.getMessage());
        }
    }
}
```

6. Troubleshooting Common Errors

1) ClassNotFoundException: Check if JDBC JAR is added to classpath.
2) Login failed for user: Check username/password and authentication settings.
3) TCP/IP connection failed: Enable TCP/IP in SQL Server Configuration Manager.